

SPEC-015: Formal Verification of Spindle Core in Lean 4

Field	Value
Document ID	SPEC-015
Title	Formal Verification of Spindle Defeasible Logic Core in Lean 4
Status	Draft
Created	2026-02-15
Author	agent:architect

1. Overview

This specification defines the scope, requirements, and acceptance criteria for formally verifying the core defeasible logic reasoning engine of spindle-rust using the Lean 4 theorem prover. The verification targets Standard Defeasible Logic (SDL) as implemented in `crates/spindle-core/`, covering both ambiguity blocking (default) and ambiguity propagation modes.

1.1 Motivation

Spindle is a reasoning engine for defeasible logic — a formalism where conclusions can be defeated by stronger evidence. Correctness of the inference algorithm is critical because:

- Downstream consumers trust conclusions.** The `hence` planner and other tools rely on spindle conclusions to schedule tasks, assign agents, and resolve conflicts. An unsound engine could silently produce wrong assignments.
- The algorithm is subtle.** The interaction between superiority, defeaters, and the two ambiguity modes creates edge cases that are difficult to validate with testing alone.
- The formal semantics are well-defined.** The proof-theoretic conditions for `+D`, `-D`, `+d`, `-d` (documented in `specs/DEFEASIBLE-LOGIC-SEMANTICS.md`) are precise enough to serve as Lean 4 theorem statements directly.

1.2 Scope

In scope (propositional SDL):

- Literal, Rule, Theory data structures (propositional fragment)
- Definite provability (`+D` / `-D`)
- Defeasible provability (`+d` / `-d`)
- Superiority relation and its role in conflict resolution
- Defeaters as asymmetric blockers
- Ambiguity blocking mode
- Ambiguity propagation mode
- Fixed-point computation algorithm
- Termination, soundness, completeness, and consistency proofs

Out of scope (deferred to future work):

- First-order variables and grounding (`crates/spindle-core/src/grounding.rs`)
- Modal operators (must/may/forbidden)
- Temporal reasoning (Allen interval algebra)
- Process mining
- Trust-weighted conclusions

- Parser correctness
- Performance properties of the worklist optimization

1.3 Reference Material

- `specs/DEFEASIBLE-LOGIC-SEMANTICS.md` — Formal semantics (proof-theoretic conditions)
 - `crates/spindle-core/src/reason.rs` — Rust implementation of the reasoning engine
 - `crates/spindle-core/src/theory.rs` — Theory and indexed theory structures
 - `crates/spindle-core/src/literal.rs` — Literal representation
 - `crates/spindle-core/src/rule.rs` — Rule types and superiority
 - Nute, D. (1994). "Defeasible Logic" — Foundational paper
 - Antoniou, G. et al. (2001). "Representation Results for Defeasible Logic" — Proof theory
-

2. User Profile

User: Formal Methods Engineer

Role: Lean 4 developer verifying spindle's core algorithms

Goals:

- Encode SDL semantics faithfully in Lean 4 type theory
- Prove that the fixed-point algorithm computes exactly the conclusions specified by the proof-theoretic conditions
- Produce machine-checked proofs that can be audited by the spindle maintainers

Constraints:

- Must use Lean 4 (not Lean 3, Coq, or Isabelle)
 - Should leverage Mathlib where appropriate (Finset, well-founded recursion)
 - Proofs must be self-contained and reproducible via `lake build`
-

3. Requirements

REQ-001: Formalize SDL Primitives

The Lean 4 formalization SHALL define inductive types for:

- `Literal` — an atomic proposition or its classical negation
- `RuleType` — strict, defeasible, defeater
- `Rule` — a labeled rule with body (list of literals), head (literal), and type
- `Theory` — a set of rules plus a superiority relation
- `ConclusionType` — the four proof tags (+D, -D, +d, -d)
- `Conclusion` — a tagged literal

These types SHALL be structurally equivalent to the Rust types in `crates/spindle-core/src/literal.rs`, `rule.rs`, and `theory.rs`, restricted to the propositional fragment (no arguments, no modes, no temporal).

Acceptance criteria: Each Lean type has a documented correspondence to its Rust counterpart. A mapping table in the Lean source comments links each Lean definition to the Rust file and line.

Trace: TEST-001

REQ-002: Formalize Complement Function

The Lean 4 formalization SHALL define a `complement` function on `Literal` such that:

- `complement(p) = ¬p`
- `complement(¬p) = p`
- `complement(complement(l)) = l` for all literals `l` (involution)

Acceptance criteria: The involution property is stated and proved as a Lean theorem.

Trace: TEST-002

REQ-003: Formalize Rule Classification Functions

The Lean 4 formalization SHALL define the rule-set functions from the semantics:

- `R(q)` — all rules with head `q`
- `Rs(q)` — strict rules with head `q`
- `Rsd(q)` — strict or defeasible rules with head `q`
- `applicable(r, proved)` — all body literals of `r` are in `proved`
- `discarded(r, disproved)` — at least one body literal of `r` is in `disproved`

Trace: TEST-003

REQ-004: Prove Definite Provability Soundness

The Lean 4 formalization SHALL state and prove:

Theorem (Soundness of +D): If the fixed-point algorithm marks literal `q` as `+D`, then `q` satisfies the proof-theoretic condition for `+D`:

```
+D q iff q ∈ Facts OR ∃ r ∈ Rs[q] : ∀ a ∈ body(r), +D a
```

Acceptance criteria: Machine-checked Lean proof. No `sorry` or `axiom` escape hatches.

Trace: TEST-004

REQ-005: Prove Definite Provability Completeness

The Lean 4 formalization SHALL state and prove:

Theorem (Completeness of -D): If the fixed-point algorithm marks literal `q` as `-D`, then `q` satisfies the proof-theoretic condition for `-D`:

```
-D q iff q ∉ Facts AND ∀ r ∈ Rs[q] : ∃ a ∈ body(r), -D a
```

Acceptance criteria: Machine-checked Lean proof. No `sorry`.

Trace: TEST-005

REQ-006: Prove Defeasible Provability Soundness

The Lean 4 formalization SHALL state and prove:

Theorem (Soundness of +d): If the fixed-point algorithm marks literal q as $+d$, then q satisfies the proof-theoretic condition for $+d$:

```
+d q iff +D q OR (
  ∃ r ∈ Rsd[q] : applicable(r)
  AND -D ~q
  AND ∀ s ∈ R[~q] : discarded(s) OR ∃ t ∈ Rsd[q] : applicable(t) AND t > s
)
```

Acceptance criteria: Machine-checked Lean proof. No `sorry`.

Trace: TEST-006

REQ-007: Prove Defeasible Provability Completeness

The Lean 4 formalization SHALL state and prove:

Theorem (Completeness of -d): If the fixed-point algorithm marks literal q as $-d$, then q satisfies the proof-theoretic condition for $-d$:

```
-d q iff -D q AND (
  ∀ r ∈ Rsd[q] : discarded(r)
  OR +D ~q
  OR ∃ s ∈ R[~q] : applicable(s) AND ∀ t ∈ Rsd[q] : discarded(t) OR -(t > s)
)
```

Acceptance criteria: Machine-checked Lean proof. No `sorry`.

Trace: TEST-007

REQ-008: Prove Termination

The Lean 4 formalization SHALL state and prove:

Theorem (Termination): The fixed-point computation terminates for any finite theory.

The proof SHALL use the argument that:

1. Each literal transitions from `Unknown` to decided ($+$ or $-$) at most once
2. Only transitions enqueue work
3. The number of literals is finite
4. Total transitions are bounded by $2 \times |\text{literals}|$ per phase

Acceptance criteria: The fixed-point function uses `Finset` or equivalent, and Lean's termination checker accepts it (possibly with a well-founded measure).

Trace: TEST-008

REQ-009: Prove Consistency

The Lean 4 formalization SHALL state and prove:

Theorem (Tag Consistency): For any theory and any literal q :

- It is never the case that both $+D\ q$ and $-D\ q$ hold
- It is never the case that both $+d\ q$ and $-d\ q$ hold

Theorem (Level Coherence): For any theory and any literal q :

- $+D\ q$ implies $+d\ q$
- $-d\ q$ implies $-D\ q$

Acceptance criteria: Machine-checked Lean proofs.

Trace: TEST-009

REQ-010: Prove Defeater Asymmetry

The Lean 4 formalization SHALL state and prove:

Theorem (Defeater Asymmetry): Defeater rules never appear in the $Rsd[q]$ set used in condition (1) of $+d$. A defeater can only block a conclusion (appear in $R[\sim q]$ in condition 3), never establish one.

Trace: TEST-010

REQ-011: Formalize Ambiguity Blocking Mode

The Lean 4 formalization SHALL encode the ambiguity blocking semantics and prove:

Theorem (Ambiguity Locality): In ambiguity blocking mode, if p and $\sim p$ are both $-d$ (ambiguous), and there exists a rule $r: \Rightarrow q$ with no body mentioning p or $\sim p$, then the provability of q is independent of the $p / \sim p$ ambiguity.

Trace: TEST-011

REQ-012: Formalize Ambiguity Propagation Mode

The Lean 4 formalization SHALL encode the ambiguity propagation semantics and prove:

Theorem (Ambiguity Infectiousness): In ambiguity propagation mode, if literal p is ambiguous (has support but is blocked by equally strong contrary), then any literal q that depends on p through an applicable rule chain also inherits the ambiguity, even if q has independent uncontested support.

Theorem (AP Subsumption): Every literal that is $+d$ under ambiguity propagation is also $+d$ under ambiguity blocking. (AP is more conservative.)

Trace: TEST-012

REQ-013: Cross-Validation Test Suite

The verification project SHALL include an executable Lean function that computes conclusions for a theory (mirroring the Rust `reason()` function), and a test suite that runs the same test cases as `crates/spindle-core/tests/` to confirm behavioral equivalence.

Acceptance criteria: At least 50 test theories from the Rust test suite are encoded as Lean `#eval` or `#check` statements, and the Lean-computed conclusions match the Rust-computed conclusions exactly.

Trace: TEST-013

4. Architecture Decisions

ADR-001: Lean 4 as Verification Target

Context: Multiple theorem provers could formalize defeasible logic (Coq, Isabelle/HOL, Agda, Lean 4).

Decision: Use Lean 4.

Rationale:

- Lean 4 has a mature standard library and Mathlib for finite set reasoning
- Lean 4's programming language is expressive enough to write an executable specification alongside the proofs
- Lean 4's `lake` build system provides reproducible builds
- Active community and documentation
- Lean 4 code can be compiled to native executables for cross-validation testing

Trade-offs:

- Mathlib is large and may increase build times
- Some proof automation is less mature than Isabelle's Sledgehammer
- Fewer existing formalizations of defeasible logic in Lean (vs. Isabelle)

ADR-002: Propositional Fragment First

Context: The full spindle-core supports first-order variables (grounding), modal operators, temporal reasoning, and trust weights. Verifying everything at once is infeasible.

Decision: Verify the propositional SDL fragment first. First-order, modal, temporal, and trust extensions are deferred.

Rationale:

- The propositional fragment captures the core algorithm complexity (fixed-point, superiority, defeaters, ambiguity modes)
- The proof-theoretic conditions in `DEFEASIBLE-LOGIC-SEMANTICS.md` are stated propositionally
- Grounding reduces first-order theories to propositional ones, so propositional correctness is foundational
- Modal/temporal extensions are compositional and can be verified incrementally later

ADR-003: Declarative Specification + Executable Model

Context: We could formalize either (a) only the proof-theoretic conditions as declarative specs and prove properties about them, or (b) also encode an executable algorithm and prove it implements the spec.

Decision: Both. Define the proof-theoretic conditions declaratively, define the fixed-point algorithm as a computable function, and prove the algorithm satisfies the conditions.

Rationale:

- Declarative-only proofs don't validate that the algorithm is correct
 - An executable model enables cross-validation against the Rust implementation (REQ-013)
 - Lean 4's `Decidable` instances and computation make executable specs natural
-

5. Test Specifications

TEST-001: Primitive Type Correspondence

Verify Lean types compile and have the expected constructors. Enumerate example values matching Rust test fixtures.

TEST-002: Complement Involution

```
theorem complement_involution (l : Literal) : complement (complement l) = l
```

TEST-003: Rule Classification

For a sample theory, verify `R`, `Rs`, `Rsd` return the expected rule sets.

TEST-004: Definite Soundness

Prove `+D` soundness for at least 3 representative theories:

- Single fact
- Strict chain ($a \rightarrow b \rightarrow c$)
- No strict rules (all `-D`)

TEST-005: Definite Completeness

Prove `-D` completeness for theories with broken strict chains.

TEST-006: Defeasible Soundness

Prove `+d` for theories with:

- Simple defeasible rule
- Competing rules with superiority
- Defeater blocking

TEST-007: Defeasible Completeness

Prove `-d` for:

- No applicable rules
- Undefeated attacker
- Symmetric conflict (no superiority)

TEST-008: Termination

Show `reason` function is accepted by Lean's termination checker (or provide explicit well-founded measure).

TEST-009: Consistency

For arbitrary theory, show `+D q ∧ -D q → False` and `+d q ∧ -d q → False`.

TEST-010: Defeater Asymmetry

Construct theory where defeater is sole rule for `q` ; show `q` is not `+d` .

TEST-011: Ambiguity Blocking Locality

Theory: `r1: => p` , `r2: => ~p` , `r3: => q` . Show `q` is `+d` despite `p / ~p` conflict.

TEST-012: Ambiguity Propagation Infectiousness

Theory: `r1: => p` , `r2: => ~p` , `r3: => q` , `r4: p => q` . Show `q` is `-d` under AP.

TEST-013: Cross-Validation

50+ theories from Rust test suite encoded as Lean test cases.

6. Observability

OBS-001: Proof Coverage Metric

Track percentage of REQ-### items with completed, `sorry` -free proofs.

OBS-002: Build Status

CI pipeline runs `lake build` and `lake test` on every commit. Build failure blocks merge.

7. Traceability Matrix

Requirement	Test	Code (Lean module)
REQ-001	TEST-001	<code>Spindle.Basic</code>
REQ-002	TEST-002	<code>Spindle.Basic</code>
REQ-003	TEST-003	<code>Spindle.Basic</code>
REQ-004	TEST-004	<code>Spindle.Definite</code>
REQ-005	TEST-005	<code>Spindle.Definite</code>
REQ-006	TEST-006	<code>Spindle.Defeasible</code>
REQ-007	TEST-007	<code>Spindle.Defeasible</code>
REQ-008	TEST-008	<code>Spindle.Termination</code>
REQ-009	TEST-009	<code>Spindle.Consistency</code>
REQ-010	TEST-010	<code>Spindle.Defeasible</code>
REQ-011	TEST-011	<code>Spindle.AmbiguityBlocking</code>
REQ-012	TEST-012	<code>Spindle.AmbiguityPropagation</code>

Requirement	Test	Code (Lean module)
REQ-013	TEST-013	Spindle.Tests